

Assignment 1: Linear Regression via Direct Optimization, Gradient Descent, and Polynomial Regression

1. Objective

This assignment focuses on regression analysis using both direct (Least Squares) and iterative (Gradient Descent) optimization methods. You will implement and compare linear regression models trained on synthetic data, and extend them to polynomial regression to study model complexity.

2. Rules and Constraints

- **Groups:** You may work on this assignment by yourself or in a group of up to 2 students.
- **Allowed Libraries:** You may use either **NumPy** or **PyTorch** for your mathematical operations.
- **Forbidden Libraries:** You are strictly prohibited from using built-in regression functions from libraries such as **scikit-learn**.
- **PyTorch Restrictions:** If you decide to use PyTorch, you must implement the functions accordingly by using PyTorch tensors instead of NumPy arrays, and standard PyTorch tensor operations instead of NumPy operations. The use of `torch.autograd` or `torch.optim` is not permitted.

3. Tasks and Grading Breakdown (100 Points Total)

The accompanying Jupyter Notebook (`hw1.ipynb`) contains the skeleton code and the data generation setup for this assignment. You are required to complete the following six parts:

Part 1: Minimum Squared Error (MSE) Evaluation (5 Points) To compare our models, we need a standard metric. The Mean Squared Error (MSE) represents the average squared difference between the predicted values ($\hat{y}$) and the actual ground truth values (y). You must implement the MSE calculation:

$$MSE = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Part 2: Linear Regression using Least Squares / Direct Optimization (10 Points)

Given input-output pairs $\{(x_p, y_p)\}_{p=1}^P$, we can find the optimal weights in a single mathematical step using the entire batch of data. This leads to the normal equation:

$$\tilde{w}^* = (\tilde{X}\tilde{X}^\top)^{-1}\tilde{X}y$$

(Note: Depending on matrix dimensions, this is often implemented as $\tilde{w}^* = (\tilde{X}^\top\tilde{X})^{-1}\tilde{X}^\top y$ in standard batch form).

Implement this analytical solution using your chosen library's linear algebra functions.

Part 3: Iterative Optimization / Batch Gradient Descent (25 Points) Instead of calculating the inverse of a matrix (which can be computationally expensive), you will iteratively update your weights to minimize the MSE. For each epoch, calculate the gradient of the loss function with respect to the weights using the full batch, and update the weights using the following rule:

$$w^{(t+1)} = w^{(t)} - \alpha \frac{2}{N} \tilde{X}^\top (\tilde{X}w^{(t)} - y)$$

Where α is the learning rate.

Part 4: Visualization (10 Points) Compare the performance (e.g., convergence behavior, accuracy, fitting quality) of both Least Squares and Gradient Descent.

1. Plot the regression line for *both* methods on the same scatter plot of the data.
2. Plot the MSE loss curve over the epochs for the Gradient Descent method.

Part 5: Polynomial Regression (30 Points) Many real-world relationships are non-linear. Polynomial regression allows us to model more complex relationships by transforming our features into polynomial terms.

1. **Creating Polynomial Features (10 pts):** For a single feature x , a polynomial of degree d creates features $[1, x, x^2, x^3, \dots, x^d]$. Implement a function to generate these polynomial features.
2. **Training Polynomial Models (20 pts):** Train polynomial regression models for degrees 1, 2, 3, 5, and 10 using the direct optimization method (normal equation). Calculate and store the MSE for each degree.
3. **Visualization and Analysis:** Run the provided visualization code to see the fitted curves for all polynomial degrees and to plot MSE vs. Polynomial Degree.

Part 6: Discussion (20 Points) In the provided Markdown cell at the end of the notebook, explicitly answer the following questions:

- **Q1:** How do the final weights and the final MSE compare between the Direct Optimization and Gradient Descent methods?
- **Q2:** If the dataset had 1,000,000 features, which method would you prefer and why? Consider the computational complexity of matrix inversion vs. iterative multiplication.
- **Q3:** Based on your polynomial regression results, which polynomial degree provides the best fit for the synthetic data? Explain your reasoning considering both MSE and visual inspection of the curves.
- **Q4:** What happens to the fitted curve as the polynomial degree increases (e.g., degree 10)? Does a lower MSE always indicate a better model? Discuss the concepts of underfitting and overfitting.

4. Submission Guidelines

- Make sure to fill in your Student Name(s) and Student ID(s) in the very first cell of the notebook.
- Ensure all code cells run sequentially without errors before submitting.
- Submit the completed `.ipynb` file through Ninova.